Claude Code: file di configurazione, memoria e contesto

> Appunti per la certificazione CCA-F. Spiegano **cosa** viene caricato, **quando** e **da quale livello** (globale aziendale, personale, team, locale), e quali chiavi accetta ogni file.
>
> ⚠️ Claude Code si aggiorna spesso e alcune chiavi possono cambiare tra una versione e l'altra. Per l'elenco completo e aggiornato fa fede la documentazione ufficiale: **docs.claude.com** → **Claude Code**.

Indice

1. I livelli di configurazione
2. Struttura tipica di una repo
3. Cosa viene letto e quando
4. Chi vince in caso di conflitto
5. In pratica per il team
6. Riferimento chiavi, file per file
7. Hook
8. Server MCP
9. Consigli rapidi e domande da esame

---

1. I livelli di configurazione

Claude Code combina configurazioni provenienti da **quattro livelli**. Ognuno ha uno scopo diverso: imporre policy aziendali, salvare preferenze personali, condividere regole col team o personalizzare un singolo progetto senza disturbare gli altri.

Livello	Dove sta	Chi lo vede	Va su Git?
Managed / Enterprise (globale aziendale)	Windows: `C:\Program Files\ClaudeCode\` macOS: `/Library/Application Support/ClaudeCode/` Linux: `/etc/claude-code/`	Tutti gli utenti della macchina / organizzazione	❌ No, lo distribuisce l'IT
User (personale, vale su tutti i progetti)	`~/.claude/` → es. `C:\Users\mauri\.claude\`	Solo tu	❌ No
Project (team)	`<repo>/CLAUDE.md` e `<repo>/.claude/`	Tutto il team	✅ Sì, si committa
Local (personale, solo questo progetto)	`<repo>/CLAUDE.local.md`, `<repo>/.claude/settings.local.json`	Solo tu	❌ No, va in `.gitignore`

A cosa serve ogni livello

- **Managed** → policy che l'utente **non può aggirare**: permessi vietati, server MCP consentiti, hook obbligatori, messaggi aziendali. Tipico in ambienti regolamentati.
- **User** → le *tue* abitudini, valide ovunque: lingua delle risposte, stile, modello preferito, skill personali che usi in ogni progetto.
- **Project** → la "verità condivisa" del team: comandi di build/test, convenzioni di codice, skill e agent specifici del progetto, permessi comuni.
- **Local** → override personali sul singolo progetto: percorsi della tua macchina, permessi extra, preferenze che non vuoi imporre ai colleghi.

> 💡 **Regola mnemonica:** più il livello è "stretto" (local, project), più è specifico; più è "largo" (user, managed), più è generale. Il *managed* però è speciale: è largo ma ha la **massima autorità**.

---

2. Struttura tipica di una repo

```
my-repo/
├── CLAUDE.md                 # istruzioni di team (committato)
├── CLAUDE.local.md           # istruzioni personali (gitignored)
├── .mcp.json                 # server MCP condivisi col team
├── .claude/
│   ├── settings.json         # permessi, hook, env di team
│   ├── settings.local.json   # override personali (gitignored)
│   ├── rules/
│   │   ├── stile.md          # sempre caricata (senza paths:)
│   │   └── api.md            # caricata solo con paths: ["src/api/**"]
│   ├── skills/
│   │   └── deploy/
│   │       ├── SKILL.md      # definizione della skill
│   │       └── script.sh     # file di supporto
│   ├── agents/
│   │   └── reviewer.md       # subagent
│   └── commands/             # vecchio formato dei comandi slash
│       └── test.md
└── src/
    └── api/
        └── CLAUDE.md         # istruzioni specifiche di sottocartella
```

Le stesse cartelle (`skills/`, `agents/`, `rules/`, `CLAUDE.md`, `settings.json`) esistono anche in `~/.claude/` per la **versione personale**, valida su tutti i progetti.

Altri file personali da conoscere:

File	Contenuto
`~/.claude.json`	Stato globale, server MCP personali (aggiunti con `claude mcp add`)

File	Contenuto
`~/.claude/projects/<progetto>/`	Trascrizioni delle sessioni e memoria automatica del progetto
`~/.claude/projects/<progetto>/memory/MEMORY.md`	Indice della memoria automatica

3. Cosa viene letto e quando

Il **contesto** (la "finestra" di token che Claude vede) è una risorsa limitata. Per questo Claude Code distingue tra ciò che è **sempre** caricato, ciò che viene caricato **su richiesta** (*lazy loading*) e ciò che **non viene mai** letto automaticamente.

3.1 All'avvio della sessione (sempre nel contesto)

Elemento	Note
`CLAUDE.md` managed	Policy aziendali
`~/.claude/CLAUDE.md`	Le tue istruzioni globali
Ogni `CLAUDE.md` / `CLAUDE.local.md` trovato **risalendo** dalla cartella di lavoro fino alla radice del filesystem	Se lanci Claude in `repo/src`, legge `repo/src/CLAUDE.md`, `repo/CLAUDE.md`, ecc.
`.claude/rules/*.md` **senza** `paths:` nel frontmatter	Regole "sempre attive"
Skill e agent: **solo nome e descrizione**	Il corpo non viene caricato (risparmio di token)
Indice della memoria automatica (`MEMORY.md`)	Solo l'indice, non i singoli file di memoria

3.2 Su richiesta (lazy)

Elemento	Quando viene caricato
`CLAUDE.md` nelle **sottocartelle**	Solo quando Claude legge/modifica file in quelle cartelle
Rules con `paths: ["src/api/**"]`	Solo quando Claude tocca file che corrispondono al glob
Corpo di una skill (`SKILL.md` + file allegati)	Quando la skill viene invocata: da te con `/nome`, oppure da Claude perché la **descrizione** corrisponde al compito
File importati con `@percorso/file.md` dentro un `CLAUDE.md`	Insieme al file che li importa (quindi *sempre*, se l'importatore è sempre caricato)

3.3 Mai letti in automatico

- `README.md`, documentazione e codice sorgente: Claude li apre **solo se gli servono** per il compito, oppure se li importi esplicitamente con `@`.
- File in cartelle diverse da quelle previste, **anche se si chiamano "rules" o "skills"**: conta il percorso, non il nome.

> 💡 **Perché è importante:** tutto ciò che è "sempre caricato" consuma token in *ogni* richiesta. Un `CLAUDE.md` gonfio riduce lo spazio utile per il lavoro e può diluire le istruzioni importanti.

---

4. Chi vince in caso di conflitto

Il comportamento cambia a seconda del **tipo** di file.

4.1 `settings.json` → si sovrascrivono per priorità

Dalla priorità **più alta** alla **più bassa**:

1. **Managed settings** (non sovrascrivibili)
2. **Argomenti da riga di comando** (es. `claude --model opus`)
3. **Local** → `.claude/settings.local.json`
4. **Project** → `.claude/settings.json`
5. **User** → `~/.claude/settings.json`

Le chiavi semplici (es. `model`) vengono sovrascritte dal livello più alto; gli array di permessi vengono **uniti** tra i livelli.

> ⚠️ **Per i permessi una regola `deny` vince sempre su `allow`, a qualsiasi livello.** Se l'utente ha `allow: Bash(rm:*)` ma il progetto ha `deny: Bash(rm:*)`, il comando è bloccato.
>
> Ordine di valutazione: `deny` → `ask` → `allow`.

4.2 `CLAUDE.md` e rules → si sommano

Non si sovrascrivono: vengono **caricati tutti insieme** nel contesto. Se due istruzioni si contraddicono non c'è un meccanismo formale di precedenza; di solito Claude privilegia quella **più specifica** (progetto o sottocartella rispetto a utente). Meglio quindi evitare contraddizioni alla radice.

4.3 Skill e agent con lo stesso nome

Quelli di **progetto** hanno la precedenza su quelli **personali**.

Riepilogo

Tipo	Meccanismo	Chi vince
`settings.json`	Override per priorità	Managed > CLI > Local > Project > User
Permessi	Unione + valutazione	`deny` sempre
`CLAUDE.md` / rules	Concatenazione	Nessuno formalmente; tende a prevalere il più specifico
Skill / agent omonimi	Sostituzione	Project > User

---

5. In pratica per il team

Da committare ✅

- `CLAUDE.md`
- `.claude/settings.json`
- `.claude/skills/`
- `.claude/agents/`
- `.claude/rules/`
- `.mcp.json`

Da aggiungere al `.gitignore` ❌

```
CLAUDE.local.md
.claude/settings.local.json
```

Buone pratiche

- **Tieni il `CLAUDE.md` corto**: comandi di build e test, convenzioni principali, struttura del progetto, "cose da non fare".
- **Sposta le cose lunghe o specifiche** in rules con `paths:` o in una skill: entrano nel contesto solo quando servono.
- **Scrivi istruzioni verificabili e concrete** ("usa `pnpm`, non `npm`") invece di generiche ("scrivi buon codice").

Comandi per controllare cosa è caricato

Comando	Cosa mostra
`/memory`	I file `CLAUDE.md` attivi (e permette di aprirli per modificarli)
`/context`	Quanto contesto occupa ogni componente
`/init`	Genera un `CLAUDE.md` iniziale analizzando la codebase

Vanno usati in un terminale `claude` interattivo.

---

6. Riferimento chiavi, file per file

6.1 `CLAUDE.md` / `CLAUDE.local.md`

Non ha chiavi: è Markdown libero che Claude legge come istruzioni. Si possono però **importare** altri file con `@`:

```
# Progetto X

Vedi @README.md per la panoramica e @package.json per i comandi npm.

## Convenzioni
- @docs/convenzioni-git.md
- Preferenze personali: @~/.claude/mie-istruzioni.md
```

Note sugli import:

- Percorsi relativi al file che importa; ammessi anche percorsi assoluti e `~`.
- Gli import possono essere annidati (con un limite di profondità).
- Gli `@` dentro blocchi di codice non vengono interpretati come import.

6.2 `.claude/rules/*.md`

Markdown più un **frontmatter opzionale**:

Chiave	Significato
`paths`	Glob dei file a cui si applica la regola. **Se manca, la regola è sempre caricata**

```
---
paths: ["src/api/**", "tests/api/**"]
---

# Regole per le API
- Ogni endpoint deve validare l'input con zod.
- Restituisci sempre errori in formato RFC 7807.
```

6.3 `.claude/skills/<nome>/SKILL.md`

Una **skill** è un pacchetto di istruzioni (più eventuali file di supporto) caricato solo quando serve.

Chiave	Significato
`name`	Nome della skill, diventa `/name`
`description`	**La più importante**: Claude decide da qui quando usarla
`argument-hint`	Suggerimento per gli argomenti, mostrato nell'autocompletamento
`allowed-tools`	Tool che la skill può usare senza chiedere il permesso
`model`	Modello da usare (`sonnet`, `opus`, `haiku`, ...)
`disable-model-invocation`	`true` = solo tu puoi lanciarla con `/nome`, Claude non la attiva da solo
`user-invocable`	`false` = la nasconde dal menu `/` (la usa solo Claude)

Chiave	Significato
`context`	`fork` = la esegue in un subagent isolato
`agent`	Quale subagent usare quando `context: fork`
`hooks`	Hook attivi solo mentre la skill è in uso

Sintassi utilizzabile nel corpo:

Sintassi	Effetto
`$ARGUMENTS` (o `$1`, `$2`, ...)	Inserisce gli argomenti passati alla skill
`!`comando``	Esegue il comando e ne inserisce l'output
`@file`	Include il contenuto di un file

Nella cartella della skill si possono aggiungere altri file (script, template, esempi) e richiamarli dal testo: vengono letti solo se servono (*progressive disclosure*).

```
---
name: release
description: Prepara una release: aggiorna CHANGELOG, versione e crea il tag.
Usala quando l'utente chiede di rilasciare o pubblicare una versione.
argument-hint: <versione>
allowed-tools: Bash(git:*), Edit
disable-model-invocation: true
---

Prepara la release $1.

Stato attuale: !`git status --short`

1. Aggiorna CHANGELOG.md seguendo @templates/changelog.md
2. Aggiorna la versione in package.json
3. Crea il tag v$1
```

6.4 `.claude/agents/<nome>.md` (subagent)

Un **subagent** è un'istanza separata di Claude con un **proprio contesto**, un proprio system prompt e propri tool. Utile per delegare compiti senza "sporcare" il contesto principale (es. review, ricerca nel codice).

Chiave	Significato
`name`, `description`	Identità e criterio con cui Claude decide di usarlo
`tools`	Tool permessi (se manca li **eredita tutti**)
`disallowedTools`	Tool vietati
`model`	`sonnet` / `opus` / `haiku` / `inherit`

Chiave	Significato
`permissionMode`	`default`, `acceptEdits`, `plan`, `bypassPermissions`, …
`skills`	Skill precaricate nel subagent
`mcpServers`	Server MCP a cui ha accesso
`hooks`	Hook propri del subagent
`maxTurns`	Limite di turni
`memory`	Memoria persistente del subagent (`user` / `project` / `local`)
`color`	Colore con cui appare nell'interfaccia

Il corpo del file è il system prompt del subagent.

```
---
name: code-reviewer
description: Revisiona il codice modificato cercando bug e problemi di sicurezza.
Usalo dopo modifiche significative.
tools: Read, Grep, Glob
model: sonnet
---

Sei un revisore esperto. Analizza solo i file modificati,
elenca i problemi in ordine di gravità e suggerisci correzioni.
Non modificare mai i file.
```

> 💡 Il subagent restituisce al chiamante **solo il risultato finale**, non tutta la sua esplorazione: è questo che fa risparmiare contesto.

6.5 `.claude/commands/*.md` (vecchio formato)

Funzionano ancora, ma ora conviene usare le **skill**. Chiavi supportate: `description`, `argument-hint`, `allowed-tools`, `model`, `disable-model-invocation`.

6.6 `settings.json` (user / project / local / managed)

Chiave	Significato
`permissions.allow` / `ask` / `deny`	Regole sui tool: `Bash(cmd:*)`, `Read(path)`, `Edit(path)`, `WebFetch(domain:x.com)`, `mcp__server__tool`
`permissions.additionalDirectories`	Cartelle extra accessibili oltre a quella di lavoro
`permissions.defaultMode`	`default`, `acceptEdits`, `plan`, `bypassPermissions`
`permissions.disableBypassPermissionsMode`	`"disable"` per vietare la modalità bypass
`model`	Modello predefinito

Chiave	Significato
`env`	Variabili d'ambiente per ogni sessione
`hooks`	Automazioni sugli eventi (vedi §7)
`disableAllHooks`	Disattiva tutti gli hook
`statusLine`	Riga di stato personalizzata
`outputStyle`	Stile delle risposte
`attribution`	Testo di attribuzione per commit e PR (sostituisce `includeCoAuthoredBy`)
`cleanupPeriodDays`	Per quanti giorni conservare le trascrizioni
`enableAllProjectMcpServers`	Approva automaticamente i server di `.mcp.json`
`enabledMcpjsonServers` / `disabledMcpjsonServers`	Server MCP approvati o bloccati uno per uno
`enabledPlugins` / `extraKnownMarketplaces`	Plugin attivi e marketplace aggiuntivi
`apiKeyHelper`	Script che restituisce la chiave API
`sandbox`	Configurazione della sandbox per i comandi Bash
`alwaysThinkingEnabled`	Ragionamento esteso sempre attivo
`companyAnnouncements`	Messaggi mostrati all'avvio (utile nei managed)

Solo nei managed settings: `allowManagedHooksOnly`, `allowManagedPermissionRulesOnly`, `strictKnownMarketplaces`. Servono a impedire agli utenti di aggiungere regole, hook o marketplace propri.

Modalità di permesso (`defaultMode`)

Modalità	Comportamento
`default`	Chiede conferma al primo uso di ogni tool sensibile
`acceptEdits`	Accetta automaticamente le modifiche ai file
`plan`	Solo analisi e pianificazione, nessuna modifica
`bypassPermissions`	Nessuna richiesta di conferma (da usare solo in ambienti isolati)

Esempio

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "model": "sonnet",
  "permissions": {
    "allow": ["Bash(npm run test:*)", "Bash(git diff:*)", "Read(src/**)"],
    "ask":   ["Bash(git push:*)"],
```

```
    "deny":  ["Read(.env)", "Read(secrets/**)", "Bash(rm -rf:*)"]
  },
  "env": { "NODE_ENV": "development" }
}
```

---

7. Hook

Gli **hook** sono comandi shell eseguiti **deterministicamente** dal harness quando si verifica un evento. A differenza delle istruzioni in `CLAUDE.md` (che Claude *dovrebbe* seguire), un hook viene eseguito **sempre**.

Eventi

Evento	Quando scatta	Uso tipico
`PreToolUse`	Prima che un tool venga eseguito	Bloccare comandi pericolosi, validare input
`PostToolUse`	Dopo l'esecuzione di un tool	Formattare il codice, lanciare il linter
`UserPromptSubmit`	Quando l'utente invia un prompt	Aggiungere contesto, filtrare prompt
`Notification`	Quando Claude invia una notifica	Notifiche desktop / Slack
`Stop`	Quando Claude termina la risposta	Lanciare i test, verifiche finali
`SubagentStop`	Quando un subagent termina	Validare il risultato del subagent
`PreCompact`	Prima della compattazione del contesto	Salvare informazioni importanti
`SessionStart`	All'avvio della sessione	Caricare contesto iniziale
`SessionEnd`	Alla chiusura della sessione	Pulizia, log

Funzionamento

- Ogni hook riceve un **JSON su stdin** con i dettagli dell'evento (es. nome del tool e parametri).
- **Exit code 0** → tutto ok, prosegue.
- **Exit code 2** → **blocca l'azione** e restituisce il messaggio di `stderr` a Claude, che può correggersi.
- Altri exit code → errore non bloccante, mostrato all'utente.
- Il `matcher` filtra i tool a cui l'hook si applica (es. `"Edit|Write"`).

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [{ "type": "command", "command": "npx prettier --write
\"$CLAUDE_FILE_PATHS\"" }]
      }
    ]
```

```
    }
  }
```

> 💡 **Istruzione vs hook:** "Formatta sempre il codice" in `CLAUDE.md` è un *suggerimento*; un hook `PostToolUse` che lancia Prettier è una *garanzia*.

---

8. Server MCP

MCP (Model Context Protocol) è lo standard con cui Claude si collega a strumenti e dati esterni (database, GitHub, Jira, …).

`.mcp.json` (server di team, nella radice della repo)

Chiave	Significato
`type`	`stdio`, `http` o `sse`
`command`, `args`, `env`	Per i server **locali** (`stdio`)
`url`, `headers`	Per i server **remoti** (`http` / `sse`)
`${VAR}` / `${VAR:-default}`	Espansione delle variabili d'ambiente: **tieni i segreti fuori dalla repo**

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/",
      "headers": { "Authorization": "Bearer ${GITHUB_TOKEN}" }
    },
    "db": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres", "${DATABASE_URL}"]
    }
  }
}
```

Scope dei server MCP

Scope	Dove sta	Come si aggiunge
Local (default)	`~/.claude.json`, legato al progetto	`claude mcp add <nome> ...`
Project	`.mcp.json` nella repo	`claude mcp add --scope project ...`
User	`~/.claude.json`, tutti i progetti	`claude mcp add --scope user ...`

Per sicurezza, i server di `.mcp.json` vanno **approvati** dall'utente al primo utilizzo (a meno di `enableAllProjectMcpServers`). Nei permessi i tool MCP si indicano come `mcp__<server>__<tool>`.

9. Consigli rapidi e domande da esame

Consigli

- ✅ Aggiungi `$schema` al `settings.json` per avere l'autocompletamento in VS Code.
- ✅ Non mettere **mai** segreti nei file committati: usa `${VAR}` e `settings.local.json`.
- ✅ Investi soprattutto sulla **`description`** di skill e agent: è da lì che Claude decide quando usarli.
- ✅ Usa gli **hook** per ciò che deve accadere *sempre*; usa `CLAUDE.md` per ciò che è *orientamento*.
- ✅ Usa **subagent** per compiti che produrrebbero molto output intermedio (ricerche, review).
- ✅ Usa **rules con `paths:`** e **skill** per non appesantire il contesto di base.